Anno Accademico 2026/2027

Leonardo Donati

Appunti

Sistemi Operativi Avanzati

Teoria e esercizi.



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Indice

1 Introduzione **4**
- 1.1 Sistema Operativo . . . 4
- 1.2 Organizzazione del Sistema di un Computer . . . 5
  - 1.2.1 Interrupt . . . 5
  - 1.2.2 Struttura della Memoria . . . 7
- 1.3 Architetture dei Computer . . . 7
- 1.4 Operazioni di un Sistema Operativo . . . 7
  - 1.4.1 Multitasking e Multiprogramming . . . 8
  - 1.4.2 Operazioni Dual-Mode . . . 8
  - 1.4.3 Timer . . . 8
- 1.5 Sicurezza e Protezione . . . 8
- 1.6 Struttura di un Sistema Operativo . . . 9

2 Chiamate di Sistema **10**
- 2.1 Application Programming Interface (API) . . . 10
  - 2.1.1 Tipi di Chiamate di Sistema . . . 11
- 2.2 Servizi di Sistema . . . 12
- 2.3 Linker e Loaders . . . 12

3 Processi **14**
- 3.1 Stato dei Processi . . . 14
- 3.2 Process Control Block (PCB) . . . 15
- 3.3 Scheduling dei Processi . . . 16
  - 3.3.1 Code di Scheduling . . . 16
  - 3.3.2 Cambio di Contesto . . . 17
- 3.4 Creazioni di Processi . . . 17
  - 3.4.1 System Calls `fork()` e `exec()` . . . 18
  - 3.4.2 Terminazione dei Processi . . . 18
- 3.5 Comunicazione fra Processi . . . 19
  - 3.5.1 Shared Memory . . . 19
  - 3.5.2 Messagge Passing . . . 20
  - 3.5.3 Sincronizzazione . . . 21
  - 3.5.4 Buffering . . . 21
  - 3.5.5 Esempi POSIX . . . 21

4 Threads **23**
- 4.1 Programmazione Multicore . . . 24
  - 4.1.1 Tipi di Parallelismo . . . 24
- 4.2 Modelli di Multithreading . . . 24

5 CPU Scheduling **26**
- 5.1 CPU e I/O Burst Cycle . . . 26
- 5.2 CPU Scheduling . . . 26
  - 5.2.1 Dispatcher . . . 27
  - 5.2.2 Criteri di Scheduling . . . 27
- 5.3 Algortimi di Scheduling . . . 28
  - 5.3.1 First-Come, First-Served (FCFS) . . . 28
  - 5.3.2 Shortest-Job-First (SJF) . . . 28
  - 5.3.3 Round-Robin (RR) . . . 29

5.3.4 Scheduling con Priorità . . . 30
5.4 Scheduling Code Multilivello . . . 30
5.5 Thread Scheduling . . . 30
5.5.1 Contention Scope . . . 31
5.6 Scheduling per Processi Multipli . . . 32
5.6.1 Approcci allo Scheduling di Processi Multipli . . . 32
5.6.2 Processori Multicore . . . 32
5.7 Scheduling della CPU in Real Time . . . 33
5.7.1 Minimizzare la Latenza . . . 33
5.7.2 Scheduling Basato su Priorità . . . 34
5.7.3 Rate-Monotic Scheduling (RMS) . . . 34
5.7.4 Ealiest Deadline First Scheduling (EDF) . . . 34
5.7.5 Proportional Share Scheduling (PSS) . . . 34
5.7.6 Scheduling Real-Time POSIX . . . 34

6 Strumenti di Sincronizzazione 36
6.1 Problema della Sezione Critica . . . 36
6.1.1 Prime Soluzioni Software . . . 36
6.1.2 Soluzione di Peternson . . . 37
6.1.3 Memory Barrier . . . 37
6.1.4 Istruzioni Hardware . . . 38

Capitolo 1

Introduzione

1.1 Sistema Operativo

> **DEFINITION** Sistema Operatvio
>
> Un sistema operatvio è un **software** che agisce come intermediario fra l'hardware e l'utente.

Ha diversi obiettivi:

- Eseguire programmi dell'utente.
- Far diventare l'uso del dispositivo conveniente.
- Usare l'hardware del computer in modo efficiente.

Per capire bene cosa fa un sistema opertivo è utili dividere la struttura di un computer in quattro parti principali:

1. **Hardware:** provvede le risorse basi, come la cpu, la memoria centrale e i dispositivi di I/O.
2. **Sistema operativo:** controlla e gestisce l'hardware per applicazioni e utenti.
3. **Programmi applicativi:** decide come le risorse hardware devono essere divise fra i vari programmi.
4. **Utenti:** agenti esterni che interagiscono con il computer.

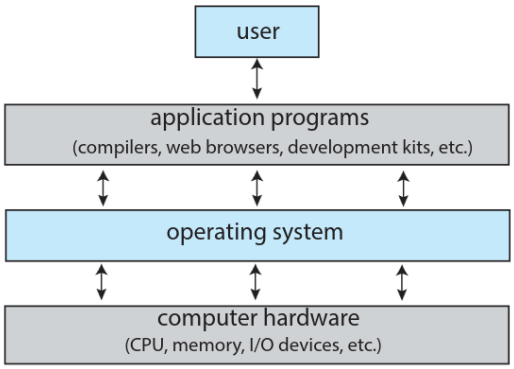


Figura 1.1: Architettura del sistema di un computer.

Dal punto di vista di un utente, deve essere facile usare applicazioni e avere buone prestazioni.
Dal punto di vista del computer, però, tutti gli utente devono essere soddisfatti, quindi il sistema operativo deve essere un **allocatore di risorse** e un **gestore di programmi**.

1.2 Organizzazione del Sistema di un Computer

Un computer moderno **general-purpose** consiste in una o più CPU e un numero di dispositivi che sono collegati tramite un **bus** comune che da accesso alla memoria condivisa e agli altri componenti.

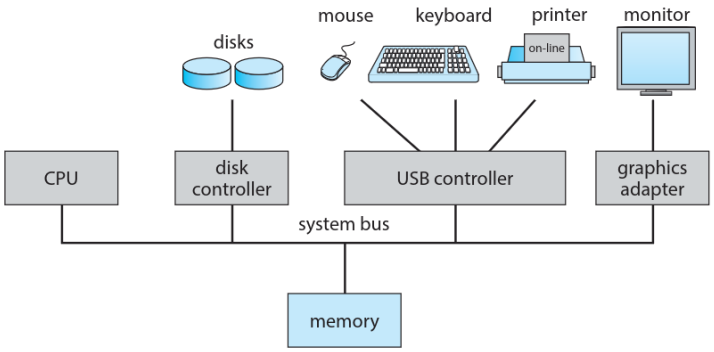


Figura 1.2: Organizzazione del sistema di un computer.

Ogni dispositivo ha un controller, che si occupa di gestire i dati fra il suo buffer interno e altri componenti. I sistemi operatvi per interagire in modo uniforme con i controller, usano dei **driver**. Dato che la CPU e i controller possono agire in parallelo, competendo per cicli di memeoria, bisogna trovare un modo ordinato per l'accesso alla memoria condivisa.

1.2.1 Interrupt

EXAMPLE Operazione di I/O

Per un'operazione di I/O si succedono diverse azioni:

- Il driver carica correttamente i registri del controller.
- Il controller guarda i regsitri per capire che operazione fare.
- Il controller carica i dati da un dispositivo al suo buffer locale.
- Il controller informa il driver di aver finito l'operazione.

Per ultimo, il driver da il controllo al sistema operativo, passando i dati letti o un puntatore a questi.

Il controller per avvisare il driver di aver finito la sua operazione, usa una **interrupt**.

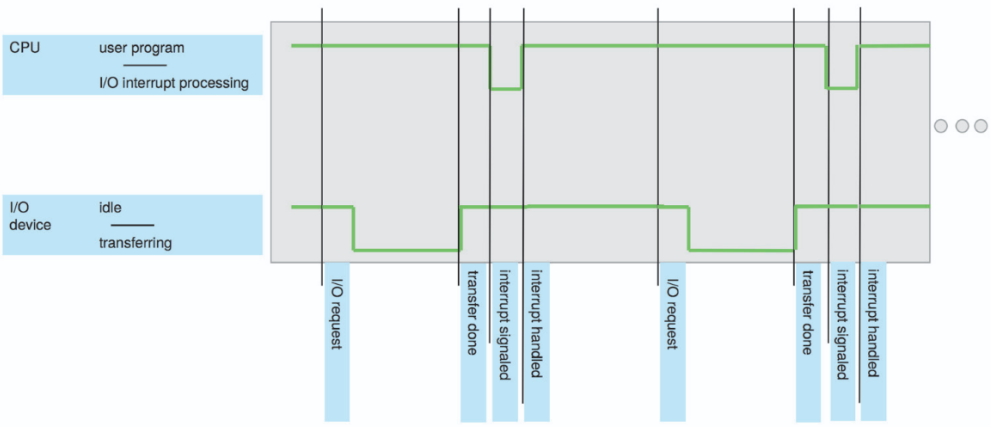


Figura 1.3: Timeline di una interrupt.

Quando la CPU riceve una interrupt, ferma immediatamente quello che sta facendo e trasferisce l'esecuzione alla service routine corretta, fino a che non è stata completata. Una volta finita, la CPU ricomincia da dove aveva interrotto, inserendo nel **program counter** l'indirizzo corretto.

> NOTA BENE
>
> Questa metodologia richiede, però, di dover salvare lo stato delle informazioni della CPU, dato che le service routine degli interrupt potrebbero modificare i registri.

Le interrupt, dato che sono molto frequenti, devono essere gestite velocemente. Per questo, si può usare una tabella, salvata nella parte basse della memoria, che contiene tutti gli indirizzi delle service routine degli interrupt, garantendo la velocità necessaria. Questo array (o **vettore degli interrupt**) di indirizzi, ha come indici dei numeri univoci che specificano la routine corretta per il tipo di interrupt.

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Figura 1.4: Tabella degli indici di un vettore di interrupt.

Implementazione

L'hardware della CPU ha un cavo chiamato **interrupt-request line**, dove la CPU riceve i segnali dai controller. Una volta intercettato il numero dell'interruzione, tramite il vettore degli interrupt, viene chiamata l'**interrupt-handler routine** corretta.
In sistemi operativi moderni, il metodo deve essere più sofisticato:

- È necessaria la possibilità di differire la gestione delle interruzioni durante l'esecuzione di elaborazioni critiche.
- È necessario un metodo efficiente per indirizzare l'esecuzione all'interrupt-handler corretto per un determinato dispositivo.
- Sono necessarie interruzioni multilivello, in modo che il sistema operativo possa distinguere tra interruzioni ad alta e bassa priorità e rispondere con il livello di urgenza adeguato.

Queste tre caratteristiche sono fornite dalla CPU e dall'**interrupt-controller hardware**.

Per gestire la priorità tra interrupt, nelle CPU esistono due linee di richieste per le interruzioni:

1. **Maskable:** può essere momentaneamente spenta dalla CPU, per dare priorità a istruzioni critiche. Questa linea è usata dai controller dei dispotivi.
2. **Non-Maskable:** usata per interrupt che non possono essere ignorate, come errori di memoria non recuperabili.

1.2.2 Struttura della Memoria

La memoria di un computer è organizzata in un gerarchia che tiene conto di tre valori:

1. **Velocità**.
2. **Capacità**.
3. **Volatilità**.

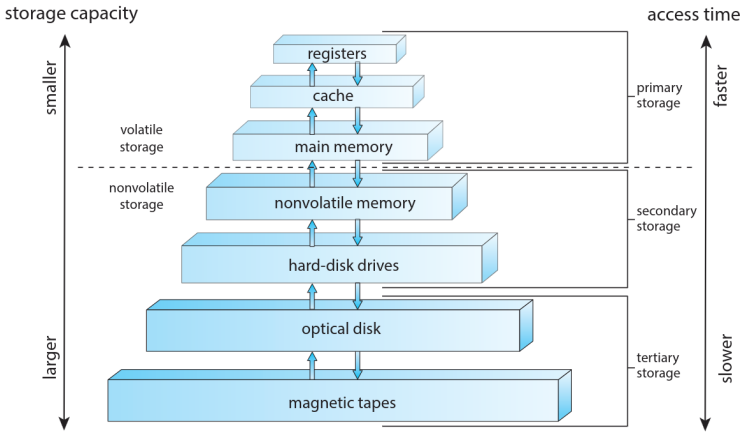


Figura 1.5: Gerarchia delle memorie.

Per spostare grandi quantità di dati, la metodologia degli interrupt descritta prima potrebbe creare molto overhead, per questo viene usata la **DMA** (**Direct Memory Access**).

> **DEFINITION** Direct Memory Access
>
> Dove aver sistemato buffer e puntatori, il controller del dispositivo sposta un grande blocco di dati dalla memoria al componente, o viceversa, senza l'intervento della CPU. Questo genera una sola interrupt per blocco, a discapito delle numerose interrupt generate senza DMA.

1.3 Architetture dei Computer

Possono esistere diverse architetture, basate sul numero di processori presenti all'interno del computer:

- **Single-Processor:** se è presente un solo processore.
- **Multi-Processor:** attualmente comune come **Multi-Core**, se sono presenti più processori. Condividono la memoria centrale e i dispositivi di I/O.
- **Cluster:** si distinguono dai computer multi-processor, perchè sono sistemi autonomi che lavorano insieme. Ogni nodo è spesso un computer multi-core.

> **DEFINITION** Alcune concetti di base
>
> **CPU:** hardware che esegue le istruzioni.
> **Processor:** chip che contiene una o più CPU.
> **Core:** unità basica della CPU.

1.4 Operazioni di un Sistema Operativo

Per avviare il computer, è necessario un programma di iniziale: il **bootstrap**. Serve a inizializzare tutte le risorse utili, come registri e i vari controller. In ultimo, deve localizzare il kernel del sistema operativo e caricarlo in memoria.

Il sistema operativo, in assenza di azione da eseguire, aspetta degli eventi, come le interrupt a livello hardware o le **trap** (**exceptions**) a livello software.

1.4.1 Multitasking e Multiprogramming

L'obiettivio del sistema operativo è tenere la CPU occupata tutto il tempo e nel mentre garantire all'utente di utilizzare più programmi contemporaneamente. Tutto questo è fattibile grazie alla capacità di essere multiprogramming, potendo gestire più programmi in esecuzione o **processi**.
Invece, il multitasking, è la capacità di poter saltare da un processo all'altro velocemente, garantendo un tempo di risposta molto basso all'utente.

1.4.2 Operazioni Dual-Mode

PROBLEMA

Il sistema operativo e gli utenti risiedono sullo stesso hardware. Si deve trovare una soluzione per evitare di eseguire codice malevolo che potrebbe alterare il funzionamento di altri programmi o del sistema stesso.

Come minimo, sono necessarie due modalità: **Kernel Mode** e **User Mode**. All'hardware è stato aggiunto un bit, chiamato **bit mode**, che permette di distinguere la modalità corrente: kernel (0) e user (1).

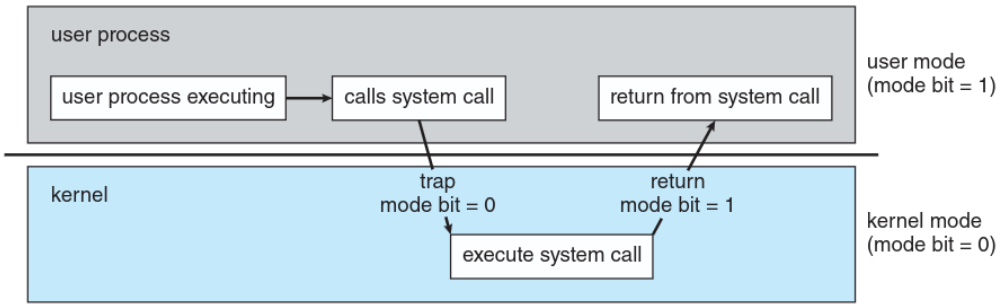


Figura 1.6: Operazione Dual-Mode.

Se si prova a eseguire una istruzione vietata in user mode, allora l'hardware lancerà una trap al sistema operativo, il quale tramite il vettore delle interrupt, prende il controllo e termina l'esecuzione del processo.

1.4.3 Timer

Bisogna assicurarsi che il sistema operativo mantenga sempre il controllo sulla CPU, per evitare che programmi in esecuzione dell'utente si blocchino in un loop infinito, oppure che una system call fallita non ritorni il controllo al sistema operativo.
Una soluzione è usare un **timer**, che interrompe il computer dopo un certo periodo. Solitamente, viene

implementato tramite un clock a frequenza fissa e un contatore. Ogni ciclo di clock, il contatore viene diminuito di una unità.

TIMER SU SISTEMI LINUX

Nei sistemi Linux, il parametro di configurazione del kernel `HZ` specifica la frequenza delle interrupt del timer. Per esempio, `HZ` = 250, significa 250 interruzioni al secondo o una ogni 4 millisecondi.

1.5 Sicurezza e Protezione

Nei sistemi operativi moderni, che supportano più utenti, i dati che vengono usati devono essere garantita solo al processo che ha ricevuto tutte le autorizzazioni.

DEFINITION Protezione

Qualsiasi meccanismo per controllare l'accesso alle risorse di un computer da parte di utenti o processi. Deve essere possibile avere un modo per specificare i controlli e controllare che vengano rispettati.

Anche se è presente un alto grado di protezione, può rimanere comunque vulnerabile a fallimenti o accessi non autorizzati.

> **DEFINITION** Sicurezza
>
> Difendere il sistema da attacchi interni ed esterni, come dos, worms, furti di dati e identità, ecc...

I sistemi operativi per distinguere fra gli utenti, usa diversi identificativi:

- **User ID:** nome e numero associato a ogni utente.
- **Group Identifier:** per definire un gruppo di utenti e associargli risorse e processi.

Per ottenere più permessi, è possibile fare **privilage escalation** per cambiare il proprio ID e avere più permessi.

1.6 Struttura di un Sistema Operativo

Capitolo 2

Chiamate di Sistema

DEFINITION Chiamata di Sistema

Interfaccia ai servizi principali, messa a disposizione da il sistema operativo. Solitamente, queste chiamate sono funzioni scritte in C/C++ o per alcuni casi specifici in linguaggio Assembly.

2.1 Application Programming Interface (API)

Nei sistemi moderni, vengono eseguite migliaia di system calls ogni secondo, quindi, gli sviluppatori, per realizzare le applicazioni si affidano a una **Application Programming Interface** (**API**).

DEFINITION API

Insieme di funzioni disponibili per un programmatore di applicazioni, inclusi i parametri passati a ciascuna funzione e i valori di ritorno che il programmatore può aspettarsi.

Le API più comuni sono quella di Windows, POSIX (Unix, Linux e MacOS) e quella per la Java Virtual Machine.

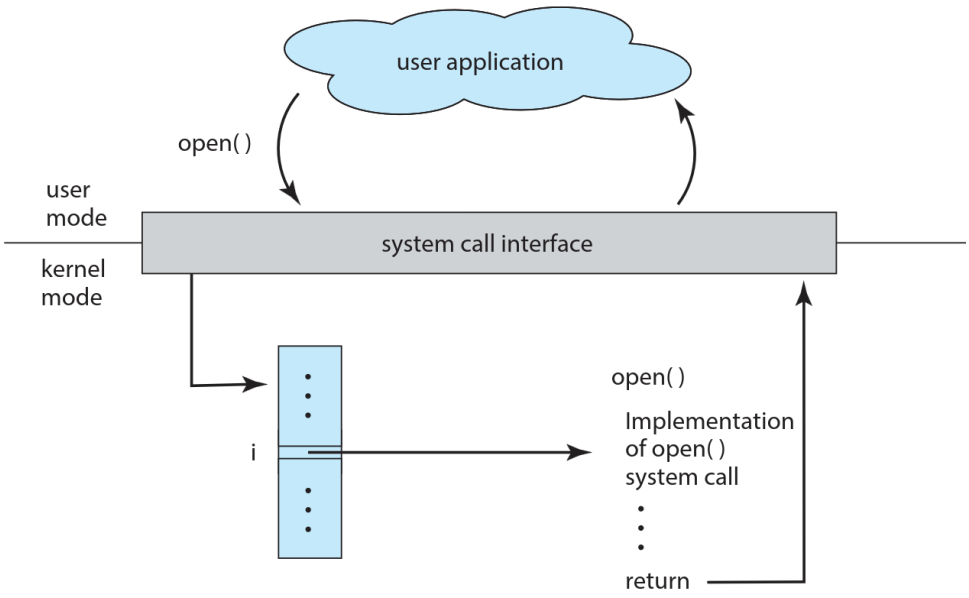


Figura 2.1: Esempio di System Call.

Un fattore nella gestione delle chiamate di sistema è l'**ambiente di esecuzione** (**RTE**), ossia l'insieme di software necessari per eseguire applicazioni scritte in un determinato linguaggio di programmazione, inclusi i relativi compilatori o interpreti, oltre ad altri componenti come librerie e loader.

L'RTE, tramite una **system call interface**, fa da link fra le system call al sistema operativo e le istruzione dell'API.
Tipicamente, a ogni chiamata di sistema viene associato un numero e viene mantenuta una tabella indicizzati secondi questi valori per gestire le call.

I parametri delle system call, vengono passati tramite registri, nel caso non sia possibile, tramite blocchi memoria oppure con una push sullo stack, dove poi il sistema oprativo eseguirà una pop.

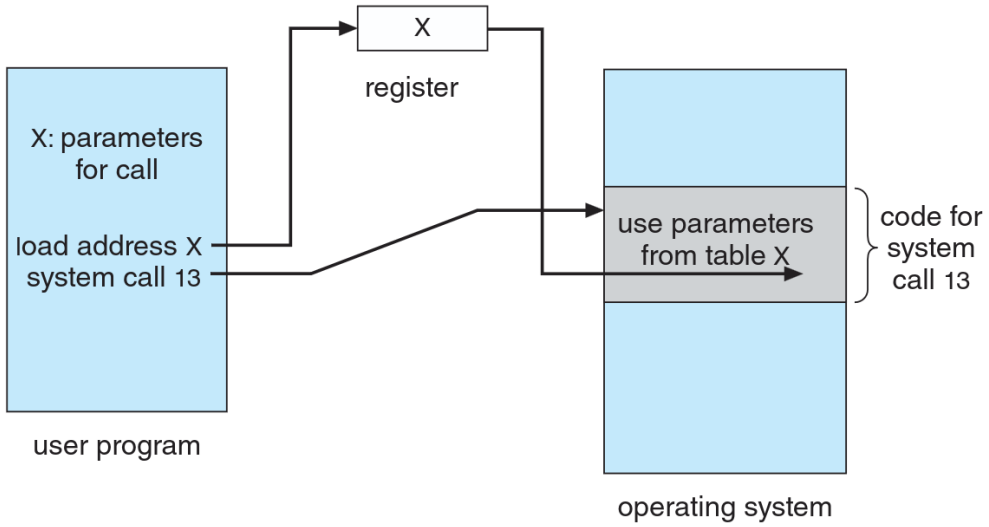


Figura 2.2: Esempio di Passaggio di Parametri.

2.1.1 Tipi di Chiamate di Sistema

Esistono sei tipi di system calls:

1. **Process Control**:
   (a) Creazione e terminazione di processi;
   (b) Caricamento ed esecuzione;
   (c) Eventi e segnali di wait;
   (d) Allocazione e liberazione della memoria;
   (e) Get e set di attributi dei processi.
2. **File Management**
   (a) Creazione ed eliminazione di file;
   (b) Apertura e chiusura di file;
   (c) Lettura e scrittura di file;
   (d) Get e set di attributi dei file.
3. **Device Management**:
   (a) Richiesta e rilascio un dispositivo;
   (b) Lettura e scrittura;
   (c) Get e set degli attributi di un dispositivi;
   (d) Attach e detach logico di dispositivi.
4. **Information Maintenance**:
   (a) Get e set di tempo e data;
   (b) Get e set di dati di sistema;
   (c) Get e set di attributi di file, processi e dispositivi.

5. **Communications**:
   (a) Creazione ed eliminazione canali di comunicazione;
   (b) Invio e ricezione di messaggi;
   (c) Trasferimento di informazioni di stato;
   (d) Attach e detach di dispositivi remoti.
6. **Protection**:
   (a) Get file permission;
   (b) Set file permission.

2.2 Servizi di Sistema

> **DEFINITION** Servizi di Sistema
>
> Offrono un ambiente conveniente per programmare ed eseguire applicazioni.
> Alcuni sono solamente warp di chiamate di sistema, altre invece sono più complessi.

Ci sono diverse categorie:

- **Gestione dei file:** permettono di eseguire diverse operazioni su file e directory.
- **Informazioni di Status:** permettono di interrogare l'SO per avere informazioni come data, tempo, spazio su memoria o dischi, ecc...
  Inoltre, permettono azioni più complesse come il logging, il debugging e l'analisi di prestazioni.
- **Modifica dei File:** modificare o creare file su dischi, tramite, per esempio, svariati text editor.
- **Supporto a linguaggi di Programmazione:** Compilatori, debugger e interpreti per linguaggi di programmazione tipo C/C++, Java, Python, ecc...
- **Esecuzione e Caricamento dei Programmi:** i programmi devono essere caricati in memoria ed eseguiti. Il sistema deve fornire linker e loader, oltre a i vari debugger.
- **Comunicazioni:** permettono di creare canali di comunicazioni virtuali fra processi, utenti o computer.
- **Servizi di Bakcground:** alcuni servizi vengono lanciati nel tempo di boot, alcuni terminano, altri invece continuano a lavorare finchè il sistema non viene spento.
  Normalmente, quest'utlimi sono chiamati **demoni**.

2.3 Linker e Loaders

Solitamente, un programma per essere eseguito da una CPU deve essere caricato nella memoria centrale e metto in lista come processo. Per fare questa cosa sono necessari diverse fasi.

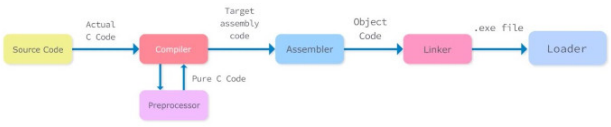


Figura 2.3: Ciclo completo di un programma C.

Per prima cosa, il programma viene compilato in un file oggetto per essere caricato in una memoria fisica. Questo formato è chiamato **file oggetto reallocabile**.

Successivamente, il **linker** combina i vari file oggetto e le librerie necessarie in un unico file eseguibile. Infine, il **loader** si occupa di caricare il programma in memoria centrale per essere eseguito dalla CPU. Un'attività importante di questa fase è la **riallocazione**, dato che è necessario assegnare un indirizzo finale alle varie parti del programma, ai dati e pure al codice.

I file oggetto e i file eseguibili presentano solitamente formati standard che includono il codice macchina compilato e una tabella dei simboli contenente metadati relativi a funzioni e variabili a cui il programma fa riferimento. Per i sistemi UNIX e Linux, questo formato standard è noto come **ELF** (**Executable and Linkable Format**).

EXAMPLE Formato ELF

Con il comando `file` è possibile identificare e valutare i formati ELF.
Per esempio con:

```
file main.o
```

verrà riportato che il `main.o` è un file ELF reallocabile.
Invece:

```
file main
```

riporterà che `main` è un file ELF eseguibile.

Capitolo 3

Processi

> **DEFINITION** Processo
>
> Un processo è un programma in esecuzione.

L'attività corrente del processo è rappresentata dal valore del **program counter** e dal contenuto dei registri. Il layout della memoria è diviso in varie parti:

- **Text Section:** codice eseguibile.
- **Data Section:** variabili globali.
- **Heap Section:** memoria allocata dinamicamente durante il tempo di esecuzione.
- **Stack Section:** spazio dei dati temporaneo quando vengono chiamate le funzioni.

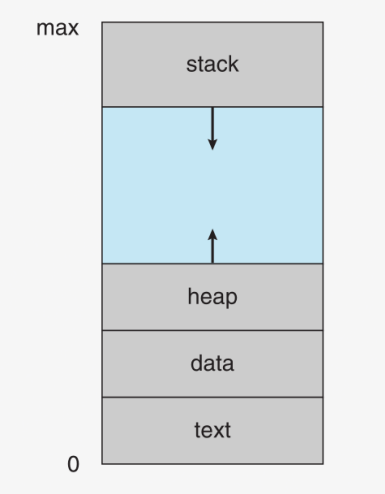


Figura 3.1: Layout Memoria.

La dimensione della sezione Data e della sezione Text sono fissate e non possono cambiare, invece, la dimensione dello Stack e dell'Heap può aumentare o diminuire dinamicamente.
Per esempio, per ogni funzione viene dichiarato un **Record di Attivazione**, con tutte i dati della funzione e viene pushato e tolto dallo stack. Invece, l'Heap può crescere in base alla memoria allocata.
Anche se queste due aree della memoria crescono una verso l'altra, il sistema operativo ha il compito di non farli mai accavallare.

3.1 Stato dei Processi

Durante la sua esecuzione, un processo può cambiare stato. Un processo può avere cinque stati differenti:

1. **New:** il processo è stato creato.

2. **Running:** le istruzione vengono eseguite.
3. **Waiting:** il processo sta aspettando l'accadimento di alcuni eventi.
4. **Ready:** il processo sta aspettando di essere assegnato a un processore.
5. **Terminated:** il processo ha finito l'esecuzione.

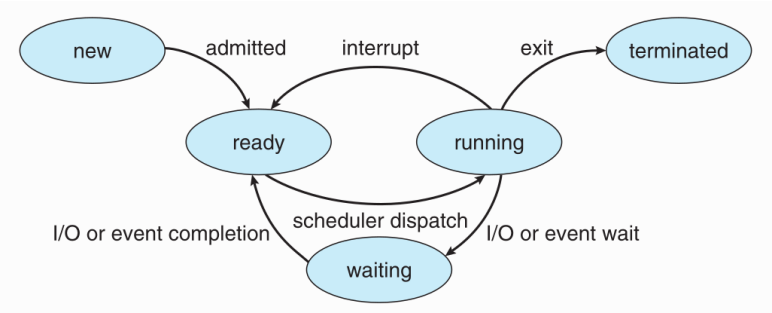


Figura 3.2: Stati Possibili di un Processo.

3.2 Process Control Block (PCB)

Ogni processo, all'interno del sistema operativo, è reppresentato da un **Process Control Block** o **Task Control Block**. Al suo interno sono contenuti tutte le informazioni associate a un processo.

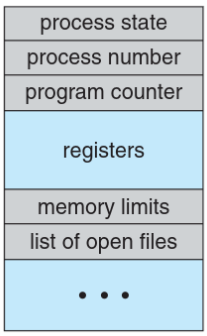


Figura 3.3: PCB.

Nello specifico, vengono specificate le seguenti informazioni:

- **Process State:** indica lo stato del processo.
- **Program Counter:** contiene l'indirizzo della prossima istruzione da eseguire.
- **CPU Registers:** contiene il tipo, il numero e lo stato dei registri usati.
- **CPU-Scheduling Information:** include i parametri per lo scheduling.
- **Memory-Managment Information:** pagine o tabelle di memoria e altre informazioni.
- **Accounting Informaztion:** quantità di tempo di CPU e tempo reale usato, job or process number, limiti di tempo e altre informazioni.
- **I/O Status Information:** lista dei dipositivi I/O parti, lista dei file aperti e altro.

All'interno dei sistemi Linux ogni PCB è reppresentato dalla structure `task_struct`, che si trova nel file `<include/linux/sched.h>` del kernel linux.

C struct task_struct

```c
struct task_struct {
  long state;                     /* state of the process */
  struct sched_entity se;         /* scheduling information */
  struct task_struct *parent;     /* this process's parent */
  struct list_head children;      /* this process's children */
  struct file_struct *files;      /* list of open files */
  struct mm_struct *mm;           /* address space */
}
```

Si può notare che all'interno della struct, oltre ai file aperti, lo stato del processo e altre informazioni, è presente pure il puntatore a una coda con i processi figli e uno che punta al processo padre.

Inoltre, sempre nei sistemi operativi Linux, sono presenti altri stati oltre a quelli classici:

- **TASK_INTERRUPTIBLE:** il task sta aspettando, può essere interrotto da un segnale.
- **TASK_UNINTERRUPTIBLE:** il task sta aspettando e non può essere interrotto da un segnale.
- **EXIT_ZOMBIE:** il processo è terminato ma le risorse devono ancora essere deallocate.
- **EXIT_DEAD:** il processo è terminato e le risorse sono state deallocate.
- **TASK_STOPPED:** il task è stato interrotto da una azione del terminale (`CTRL + Z`).
- **TASK_TRACED:** il task è seguito da un altro processo per un esecuzione passo passo.

C /include/linux/sched.h

```c
/* Used in tsk->__state: */
#define TASK_RUNNING 0x00000000
#define TASK_INTERRUPTIBLE 0x00000001
#define TASK_UNINTERRUPTIBLE 0x00000002
#define __TASK_STOPPED 0x00000004
#define __TASK_TRACED 0x00000008
#define EXIT_DEAD 0x00000010
#define EXIT_ZOMBIE 0x00000020
#define EXIT_TRACE (EXIT_ZOMBIE | EXIT_DEAD)

/* Used in tsk->__state again: */
#define TASK_PARKED 0x00000040
#define TASK_DEAD 0x00000080
#define TASK_WAKEKILL 0x00000100
#define TASK_WAKING 0x00000200
#define TASK_NOLOAD 0x00000400
#define TASK_NEW 0x00000800
```

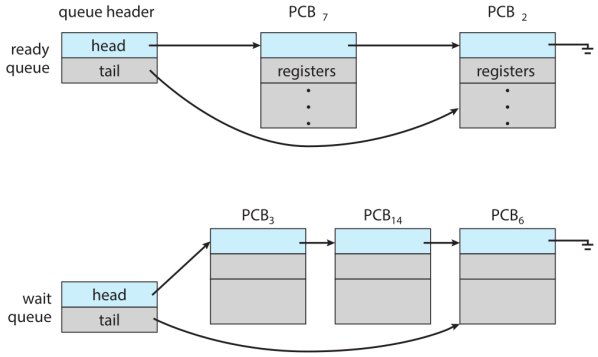
3.3 Scheduling dei Processi

Per una CPU con un solo core, non potrà mai esserci più di un processo in esecuzione. Invece, in caso di una archiettura multicore, i processi dovranno essere schedulati e aspettare un core libero. Il numero di processi in memoria è detto **grado di multiprogrammazione**.

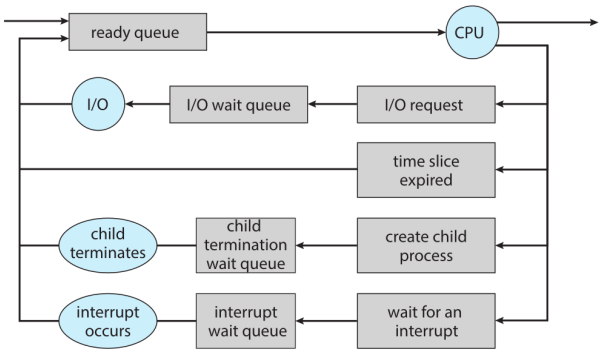
3.3.1 Code di Scheduling

Quando i processi entro nel sistema, vengono inseriti in una coda, la **Ready Queue**. Questa coda è una linked-list, l'intestazione contiene il puntatore al primo PCB e ogni altro PCB contiene il puntatore al prossimo task della coda.
Esistono anceh altre code, nello specifico, se per esempio un processo deve fare delle lunghe operazioni di I/O, allora il processo che dovrà aspettare verrà messo nella **Wait Queue**.
Un ottimo modo per rappresentare lo scheudling di un processo è tramite un diagramma.

Per prima cosa, un processo viene inserito nella coda ready dove aspettare di essere eseguito, successivamente,in base alla circostanze, può essere messo nel coda di wait o di nuovo nella coda di ready.



(a) Code Ready e Wait.



(b) Diagramma dello Scheudling di un Processo.

Questo ciclo viene eseguito finchè il processo non termina. In tal caso, se il processo ha finito la sua esecuzione, verrà rimosso da ogni coda e le risorse del PCB verranno deallocate.

3.3.2 Cambio di Contesto

In sistemi moderni general-purpose, la frequenza di interrupt che richiedono una routine del kernel per essere gestiti, può essere elevata. Ogni volta che accade, deve essere salvato il **contesto** del processo in esecuzione sul core, per poterlo ripristinare successivamente. Tutti questi dati sono salvati all'interno del PCB, permettendo quindi due azioni: **save state** e **restore state**.

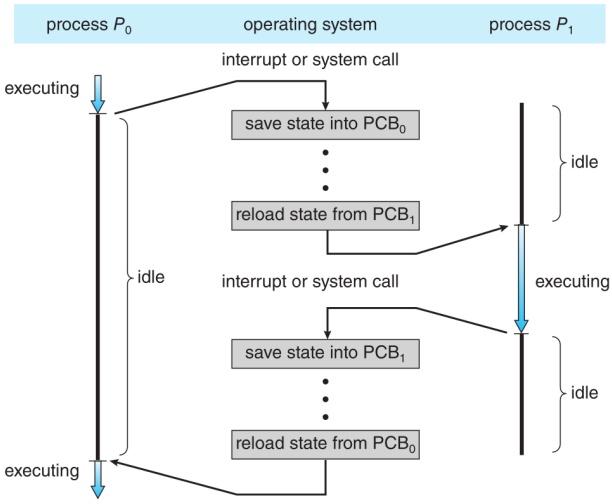


Figura 3.5: Context Switch.

Il tempo per il cambio di contesto è puro overhead, dato che il sistema non fa nulla di utile in questi millisecondi. Più è alta la complessità del sistema e del PCB più tempo sarà necessario. La velocità delle memorie può aiutare a ridurre il tempo di cambio di contesto.

3.4 Creazioni di Processi

I processi possono creare a loro volta altri processi figli, che a loro volta ne possono creare altri. Questo porta alla creazione di un labero dei processi. Ogni processo è indificato dal **Process Identifier** (**PID**), che di solito è un integer. Alla creazione, ci possono essere diverse opzioni:

- **Condivisione delle Risorse:**
  - Genitore e figlio condividono le stesse risorse.
  - Il figlio condivide un sottoinsieme delle risorse del genitore.

  - Genitore e figlio non condividono nessuna risorsa.

- **Esecuzione:**
  - Il processo figlio e quello genitore eseguono contemporaneamene.
  - Il genitore aspetta che il figlio termini.

- **Spazio di Indirizzamento:**
  - Il figlio è una copia duplicata del padre.
  - Viene caricato un altro programma all'interno del figlio.

3.4.1 System Calls `fork()` e `exec()`

Nei sistemi Linux, per creare un nuovo processo può essere usata la funzione `fork()`. Con questa istruzione, viene creato un nuovo processo figlio, con lo stesso spazio di indirizzamento del padre. Al suo interno può essere eseguito anche un altro programma, tramite la funzione `exec()`.
I due processi inizialmente eseguono contemporaneamene, nello specifico, il processo figlio avrà un valore di ritorno di `fork()` con valore uguale a zero, invece, il genitore avrà un valore diverso da zero.

C Esempio di utilizzo di fork() e exec()

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
  pid_t pid;

  /* fork a child process */
  pid = fork();
  if (pid < 0) { /* error occurred */
    fprintf(stderr, "Fork Failed");
  return 1;
  }
  else if (pid == 0) { /* child process */
    execlp("/bin/ls","ls",NULL);
  }
  else { /* parent process */
    /* parent will wait for the child to complete */
    wait(NULL);
    printf("Child Complete");
  }
}

return 0;
```

3.4.2 Terminazione dei Processi

Normalmente, un processo esegue tutte le istruzioni e poi chiede al sistema operativo di interromperlo con la system call `exit()`. In questo caso, le risorse sono deallocate e viene ritornato un valore al genitore che sta aspettando con la system call `wait()`.
In alcuni casi, è possibile terminare anche con la system call `abort()`.

Se viene terminato un processo genitore, allora anche tutti i figli verranno terminati, altrimenti si possono verficare situazioni problematiche:

- Se il genitore non usa l'istruzione `wait()`, allora i processi figli saranno chiamati **zombie**.
- Se il genitore si interrompe prima di chiamare l'istruzione `wait()`, allora i processi figli saranno **orfani**.

3.5 Comunicazione fra Processi

All'interno di un sistema, i processi possono essere indipendenti oppure possono cooperare. In quest'ultimo caso, necessitano di un modo per comunicare, ossia una **Interprocess Comunication** (**IPC**).
Le modalità possono essere due: **shared memory** o **message passing**.

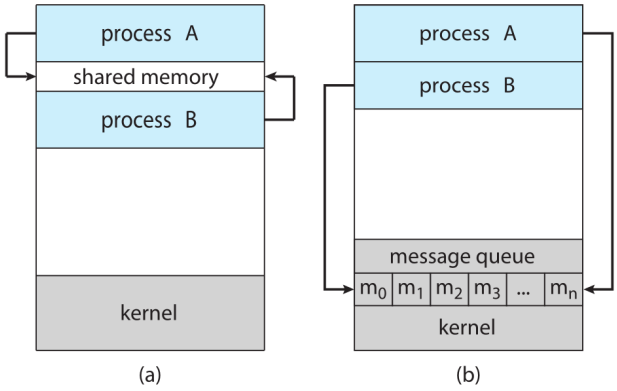


Figura 3.6: Diverse Modalità di Comunicazione.

INTUITION Problema Produttore-Consumatore

Processo **produttore** produce informazione e il processo **consumatore** la consuma.
Può avere due variazioni:

- **Unbounded-Buffer:** non c'è alcun limite pratico alla dimensione del buffer. Il produttore non attende mai. Il consumatore attende se non c'è alcun buffer da consumare.
- **Bounded-Buffer:** presuppone che vi sia una dimensione fissa del buffer. Il produttore deve attendere se tutti i buffer sono pieni. Il consumatore attende se non vi è alcun buffer da consumare.

3.5.1 Shared Memory

Un modo per comunicare fra processi è creare un'area condivisa in cui vengono scritte e lette informazioni. Il controllo è lasciato ai processi utenti e necessita di un meccanismo di sincronizzazione per condinare l'accesso alla memoria.

Per esempio, si potrebbero inserire le seguenti variabili in un'area di memoria condivisa:

C Variabili nella Shared Memory

```c
#define BUFFER_SIZE 10

typedef struct {
  . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

Questo metodo è limitato a `BUFFER_SIZE` - 1 elementi.
Si può creare un produttore che scriva dati all'interno di una lista buffer:

C Producer

```c
item next_produced;
```

```c
while (true) {
  /* produce an item in next produced */
  while (((in + 1) % BUFFER_SIZE) == out)
    ; /* do nothing */

  buffer[in] = next_produced;
  in = (in + 1) % BUFFER_SIZE;
}
```

Infine, si può usare un consumatore per leggere i dati da una lista di buffer:

C Consumer

```c
item next_consumed;

while (true) {
  while (in == out)
    ; /* do nothing */

  next_consumed = buffer[out];
  out = (out + 1) % BUFFER_SIZE;
  /* consume the item in next consumed */
}
```

Se si volesse creare una soluzione a questo problema che riesca a riempire tutti i buffer, si può usare un **contatore**. Partendo da 0, quando un buffer viene riempito, il consumatore aumenta di una unità il contatore, invece, quando il consumatore svuota un buffer, il contatore viene diminuito di una unità. Questa soluzione può funzionare, ma se le operazioni non sono atomiche, si può creare una **Race Condition**.

EXAMPLE Race Condition

S0: [producer] register1 = counter — *register1=5*
S1: [producer] register1 = register1 + 1 — *register1=6*
S2: [consumer] register2 = counter — *register2=5*
S3: [consumer] register2 = register2 - 1 — *register2=4*
S4: [producer] counter = register1 — *counter=6*
S5: [consumer] counter = register2 — *counter=4*

3.5.2 Messagge Passing

Un altro modo per far comunicare i processi può essere un meccanismo di sincronizzazione per il passaggio di messaggi, senza condividere una stessa area di memoria. In casi in cui i processi siano in computer diversi o reti diverse, questa modalità può risultare molto utile.
Per fare questo servono due operazioni principali: `send(message)` e `receive(messagge)`. Inoltre, serve anche un **canale di comunicazione**, che può avere diverse applicazioni:

- Comunicazioni dirette e indirette.
- Comunicazioni sincrone e asincrone.
- Buffering automatico o esplicito.

Comunicazione Diretta

In questo caso, il link di comunicazione viene stabilito automaticamente fra una coppia di processi e può essere usato solo da essa. In questo caso si possono avere due modalità:

1. **Simmetrica:** entrambi i processi devono nominare l'altro processo per comunicare.
2. **Asimmetrica:** solo chi invia deve nominare il destinatario per comunicare, ma non viceversa.

Comunicazione Indiretta

In questo caso i messaggi vengono inviati a una **mailbox** o **porta**. A livello più astratto può essere visto come un oggetto in cui i messaggi possono essere aggiunti o rimossi. Ogni mailbox dovrà avere il suo numero identificativo.
In questo caso il link di comunicazione si può stabilire solo se entrambi i processi hanno la stessa mailbox, un link può essere associato a più processi.

3.5.3 Sincronizzazione

La comunicazione avviene tramite le primitive `send()` e `receive()` e ci sono diversi design per implementarle:

- **Blocking:** è considerata sincrona. Quindi:
  - **Blocking send:** il processo mittente viene bloccato finché il messaggio non viene ricevuto dal processo destinatario o dalla casella di posta.
  - **Blocking receive:** il ricevente rimane in attesa finché non è disponibile un messaggio.
- **Non Blocking:** è considerata asincrona. Quindi:
  - **Non blocking send:** il processo di invio trasmette il messaggio e riprende il funzionamento.
  - **Non blocking receive:** il ricevente recupera un messaggio valido oppure un valore nullo.

Ci possono essere diverse combinazioni di `send()` e `receive()`, se entrambi sono bloccanti si chiama **rendevouz**.

3.5.4 Buffering

Se la comunicazione è diretta o indiretta, i messaggi scambiati tra processi comunicanti risiedono in una coda temporanea. Queste code possono essere implementate in tre modi:

1. **Capacità zero:** lunghezza pari a zero. Il mandante deve aspettare che il ricevente abbia ricevuto il messaggio (rendevouz).
2. **Capacità limitata:** la lunghezza della coda può contenere n messaggi. Se la coda ha spazio, i messaggi vengono mandati e inseriti, altrimenti, se piena, il mandante deve fermarsi.
3. **Capicità non limitata:** lunghezza della coda praticaticamente illimitata, il mandante può continuare a mandare messaggi senza aspettare.

3.5.5 Esempi POSIX

C Esempio di Producer POSIX

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main(){
  /* the size (in bytes) of shm obj */
  const int SIZE = 4096;

  /* name of the shared memory object */
  const char *name = "OS";

  /* strings written to shared memory */
  const char *message_0 = "Hello";
  const char *message_1 = "World!";
```

```c
  /* shared memory file descriptor */
  int fd;

  /* pointer to shared memory obect */
  char *ptr;

  /* create the shared memory object */
  fd = shm_open(name, O_CREAT | O_RDWR, 0666);

  /* configure the size of the shared memory object */
  ftruncate(fd, SIZE);

  /* memory map the shared memory object */
  ptr = (char *)
  mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

  /* write to the shared memory object */
  sprintf(ptr,"%s", message_0);
  ptr += strlen(message_0);
  sprintf(ptr,"%s", message_1);
  ptr += strlen(message_1);

  return 0;
}
```

C Esempio POSIX Consumer

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main(){
  /* the size (in bytes) of shared memory object */
  const int SIZE = 4096;

  /* name of the shared memory object */
  const char *name = "OS";

  /* shared memory file descriptor */
  int fd;

  /* pointer to shared memory obect */
  char *ptr;

  /* open the shared memory object */
  fd = shm_open(name, O_RDONLY, 0666);

  /* memory map the shared memory object */
  ptr = (char *)
  mmap(0, SIZE, PROT_READ, MAP_SHARED, fd, 0);

  /* read from the shared memory object */
  printf("%s",(char *)ptr);

  /* remove the shared memory object */
  shm_unlink(name);

  return 0;
}
```

Capitolo 4

Threads

Normalmente, la maggior parte delle applicazioni che funzionano nei sistemi moderni, sono implementate come un processo separato che ha controllo di più **thread**.

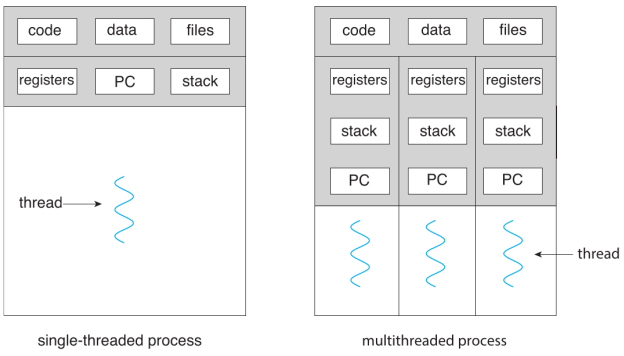


Figura 4.1: Singlethread VS Multithread.

Le applicazioni, possono venire programmate pure per essere **multicore**.
Per esempio, un server web in ascolto, potrebbe creare un nuovo processo per ogni richiesta che riceve, ma se a loro volta i processi ne creano altri, si potrebbe avere un overhead molto elevato. Una soluzione è creare un thread che si mette in ascolto e, per ogni richiesta, crearne uno nuovo.

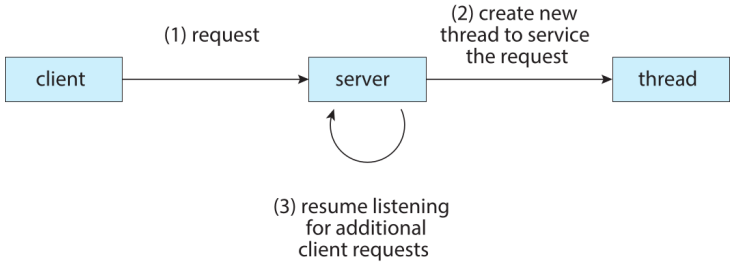


Figura 4.2: Esempio di Utilizzo di Thread.

Utilizzare i thread garantisce molti benefici:

- **Reattività:** permette di fare operazioni, anche se una parte di un processo è ferma. Molto utile per le interfacce grafiche.
- **Condivisione delle Risorse:** i processi necessitano di tecniche particolari per condividere le risorse, i thread, invece, condividono la memoria con il processi che li ha creati.
- **Economia:** creare nuovi thread e cambiare il contesto fra di loro occupa meno tempo e meno memoria. L'overhead introdotto è minore.
- **Scalabilità:** può trarre benficio molto dalla architetture multicore, eseguendo più thread contemporaneamente su più core.

4.1 Programmazione Multicore

I sistemi multicore consistono nell'inserire molteplici **core** di calcolo in un unico chip di elaborazione, in cui ciascun core appare al sistema operativo come una CPU distinta. La programmazione multithread offre un meccanismo per un utilizzo più efficiente di questi core di calcolo, consentendo una maggiore concorrenza.

In un sistema singlecore, avere più thread permette di avere **concorrenza**, invece, in un sistema multicore, permette di ottenere il **parallellismo**.

NOTA BENE

Concorrenza $\neq$ Parallellismo
Un sistema concorrente supporta più di un'attività, consentendo a tutte le attività di avanzare. Al contrario, un sistema parallelo è in grado di eseguire più di un'attività simultaneamente.

DEFINITION Legge di Amdahl

La legge di Amdahl è una formula che individua i potenziali incrementi di prestazioni derivanti dall'aggiunta di ulteriori core a un'applicazione che presenta sia componenti sequenziali che paralleli.

$$\text{speedup} \leq \frac{1}{S + \frac{(1-S)}{N}}$$

Dove S rappresenta la parte di calcolo sequenziale e N il numero di core.

4.1.1 Tipi di Parallelismo

Generalmente ci sono due tipi di parallellismo:

1. **Parallelismo dei Dati:** cerca di dividere un insieme di dati in più sottoinsiemi e distribuirli su più core.
2. **Parallelismo dei Task:** distribuire i thread sui core, ognuno eseguirà un suo task.

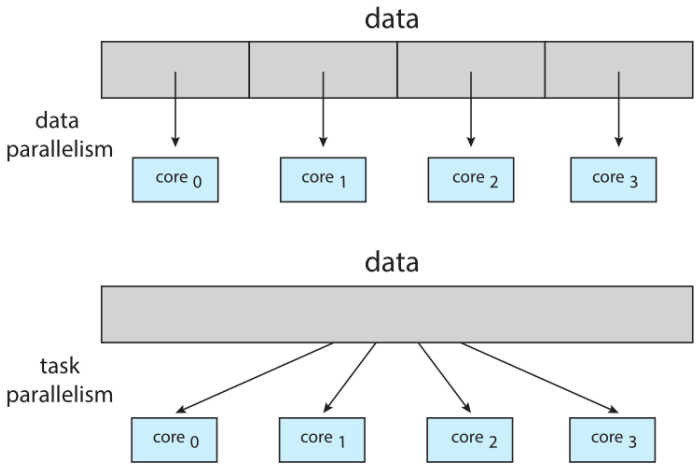


Figura 4.3: Tipi di Parallelismo.

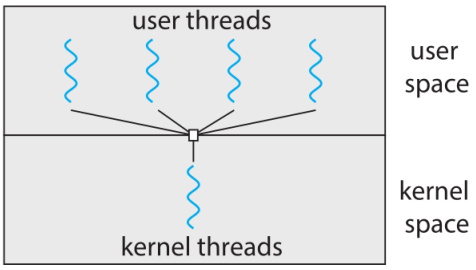
4.2 Modelli di Multithreading

Esistono due tipi di thread:

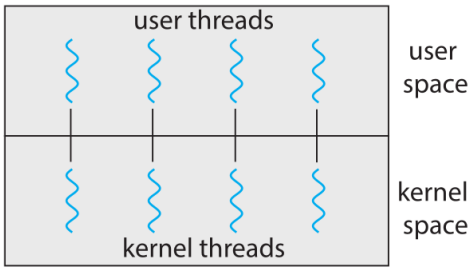
1. **User Thread:** gestiti da librerie nel livello utente (`pthread`, Windows Thread e Java Thread).
2. **Kernel Thread:** supoortati del kernel di tutti i maggiori sistemi operativi.

Inoltre, esistono tre modelli diversi per mappare i thread:

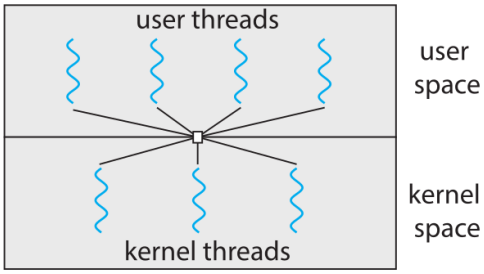
1. **Many-To-One:** più thread user-level vengono mappati a un singolo kernel thread.
2. **One-To-One:** ogni thread user-level viene mappato a un kernel thread.
3. **Many-To-Many:** più thread user-level possono essere mappati su più kernel thread.



(a) Modello Many-To-One.



(b) Modello One-To-One.



(c) Modello Many-To-Many.

Capitolo 5

CPU Scheduling

L'obiettivo della multiprogrammazione è fare in modo che vi sia sempre un processo in esecuzione, al fine di massimizzare l'utilizzo della CPU.

5.1 CPU e I/O Burst Cycle

L'esecuzione di un processo consiste in un ciclo di utilizzo della CPU e attesa di operazioni di I/O. Inizia con un periodo di utilizzo della CPU (**CPU burst)**, seguito da un periodo di attesa per operazioni di I/O (**I/O burst**), continuando così finchè la CPU termina con una system call per concludere l'esecuzione.

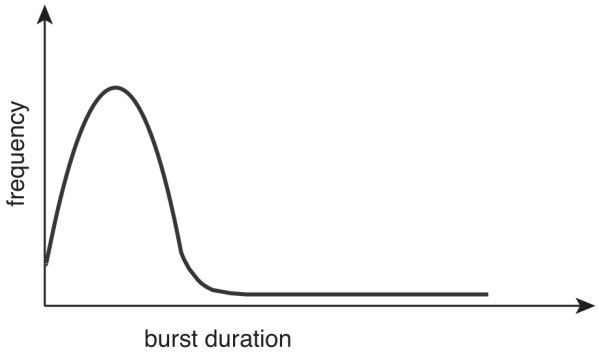


Figura 5.1: Durata del burst della CPU.

5.2 CPU Scheduling

Ogni volta che la CPU diventa inattiva, il sistema operativo deve selezionare uno dei processi nella ready queue affinché venga eseguito. Questo compito è assegnato allo **Scheduler dello CPU**. Gli elementi contenuti in tali code sono, in genere, i blocchi di controllo del processo (PCB) dei processi stessi.
Ci possono essere quattro circostanze diverse:

1. Processo passa da stato di runnig a stato di waiting.
2. Processo passa da stato di running a stato ready.
3. Processo passa da stato di waiting a stato ready.
4. Processo che termina.

Per il caso 1 e 4 lo scheduling è chiamato **nonpreemtive**, dato che non c'è nessun tipo di scelta da fare. Invece, nei casi 2 e 3, lo scheduling è chiamato **preemtive**.

5.2.1 Dispatcher

DEFINITION

Componente che fa parte della funzione di scheduling della CPU. Questo modulo si occupa di dare il controllo del processo selezionato dallo scheduler al core disponibile.

Ci sono diverse funzioni che esegue:

- Cambio di contesto fra un processo e l'altro.
- Cambiare alla modalità utente.
- Saltare alla posizione corretta per riprendere l'esecuzioni di un programma.

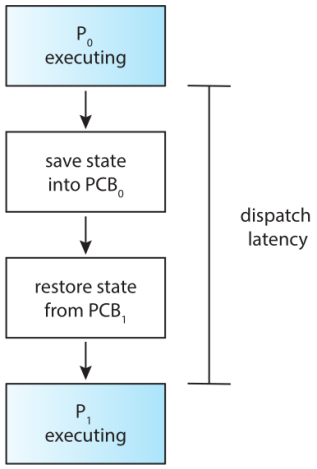


Figura 5.2: Dispatcher e Latenza.

Deve essere molto veloce e efficente, dato che deve essere chiamato a ogni context swicth.
Il tempo che impiega il dispatcher è chiamato **latenza del dispatcher**.

5.2.2 Criteri di Scheduling

Ci sono numerosi algortimi di scheduling, ognuno che migliora e ottimizza diversi aspetti. Per confrontarli, è necessario avere dei criteri precisi:

- **Utilizzo della CPU:** mantenere la CPU più occupata possibile.
- **Throughput:** massimizzare il numero di processi che completano la propria esecuzione per unità di tempo.
- **Tempo di esecuzione:** minimizzare il tempo totale di esecuzione di un processo (ready queue + i/o and cpu time).
- **Tempo di waiting:** minimizzare il tempo che un processo rimane nella ready queue.
- **Tempo di risposta:** minimizzare il tempo il tempo che il processo ci mette, dopo essere stato eseguito, a dare la prima risposta.

In base al tipo di utilizzo, si può cercare di ottimizzare alcuni criteri rispetto ad altri.
Per esempio, un computer consumer, dovrà preferire velocità e fluidità nell'utilizzo di interfaccie e programmi.
In un server, invece, potrebbe essere necessario ottimizzare il throughput, dato che non ha bisogno di interfaccie grafiche fluide.

5.3 Algortimi di Scheduling

In questa parte verrano analizzati i principali algortimi di scheduling. In questa prima parte, viene considerato il caso in cui la CPU abbia un solo core, potendo eseguire un solo processo alla volta.

5.3.1 First-Come, First-Served (FCFS)

Algortimo più semplice, viene gestito tramite una coda FIFO. Quando un processo viene inserito nella ready queue, il suo PCB viene aggiunto in coda. Quando la CPU è libera viene eseguito il primo processo della coda.

Process	Burst Time
P_1	24
P_2	3
P_3	3



Figura 5.3: Gaant FCFS.

Questo metodo è nonpreemtive, quindi una volta che un processo viene asseganto alla cpu, finisce di eseguire. Questo è particolarmente scomodo in sistemi interattivi, in cui ogni processo ha una fetta di tempo di cpu. Inoltre, nel caso il primo processo sia quello più lungo, potrebbe aumentare sostanzialmente il tempo medio di wait, risultando in un utilizzo della cpu poco efficente.

5.3.2 Shortest-Job-First (SJF)

In questo algoritmo, a ogni processo viene assegnato il tempo di cpu burst. Quando la cpu diventa libera, viene mandato in esecuzione il processo con il tempo minore.

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

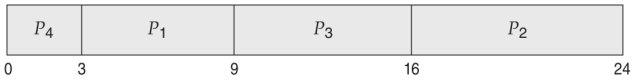


Figura 5.4: Gaant SJB.

Se due processi avranno un burst time uguale, allora si usa fra di loro la metodologia FCFS.
Questo algortimo sarebbe ottimale, ma non è implementabile, visto che non si può sapere a priori quale sarà il tempo di burst. L'unico modo è provare a prevederlo. Il prossimo tempo di burst della cpu è predetto attarverso una **media esponenziale**, utilizzando il burst precedente:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

Dove:

- $0 \geq \alpha \geq 1$.
- t_n è il burst precedente.
- τ_n contiene tutti i valori passati.
- τ_{n+1} è il tempo che vogliamo predirre.

Se α è uguale a 0, allora la storia passata non ha importanza, invece, se α è uguale a 1, allora ha importante solo il burst più recente.
Solitamente, come valore di α viene utilizzato $\frac{1}{2}$ e come valore di τ_0 viene utilizzata una costante o la media generale del sistema.

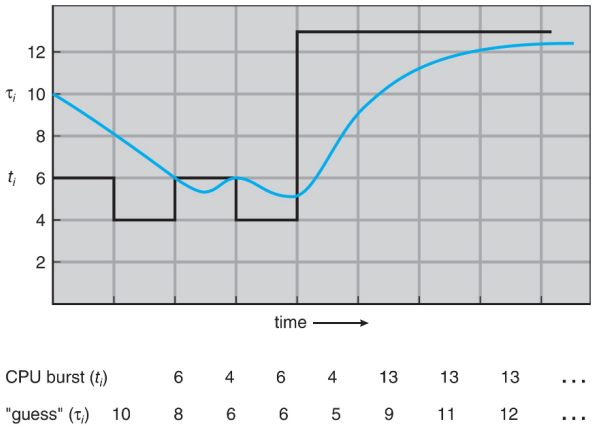


Figura 5.5: Prediction burst SJB.

In questo caso con i valori $\alpha = \frac{1}{2}$ e $\tau_o = 10$, si può raggiungere una buona approssimazione.
Questo algortimo può essere sia non nonpreemtive che preemtive, nel caso i processi abbiamo un burst di cpu minore del tempo totale che gli serve. Quest'ultimo caso è anche chiamato **shortest-remaining-time-first**.

5.3.3 Round-Robin (RR)

Questo algortimo è simile al FCFS, ma è preemtive, permettendo al sistemi di cambiare processi.
L'implemntazione richiede l'utilizzo di una piccola unità di tempo chiamata **time quantum** o **time slice**.
Nello specifico, la coda ready viene trattata come se fosse circolare e lo scheduler esegue per un quantum time ogni processo, uno dopo l'altro. Anche in questo caso la ready queue è trattata come FIFO, quindi i nuovi processi vengono aggiunta alla fine. Le situazioni possibili sono due:

1. Il processo dura meno di un quantum time, quindi termina e si passa a quello successivo.
2. Il processo dura di più di un quantum time, quindi quando il timer finisce, processo corrente viene inserito alla fine della coda, viene lanciata una interrupt al sistema operativo e viene eseguito un cambio di contesto al processo successivo.

Process	Burst Time
P_1	24
P_2	3
P_3	3

Ipotizziamo un quatum time q uguale a $4ms$. Il processo P_1 viene eseguito per 4 millisecondi, poi interrotto. Dopo il cambio di contesto viene eseguito P_2 che termina. Per P_3 stessa situazioni. Infine, rimane da eseguire il processo P_1.

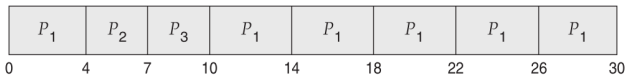


Figura 5.6: Gaant RR.

Se il quatum time è molto elevato, questo algortimo si trasforma in un FCFS, invece, se molto piccolo, potrebbe richiedere un numero molto grande di cambi di contesto. Solitamente, l'80% del tempo di burst deve essere inferiore al quatum time per ottenere un buon risultato.

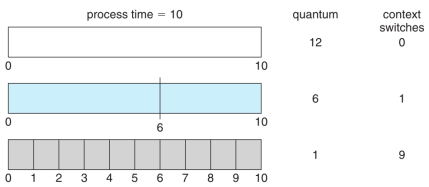


Figura 5.7: Esempio di scelte del quatum time.

5.3.4 Scheduling con Priorità

L'algortimo SJF era una caso specifico del più generale algortimo di scheduling con priorità. A ogni processo viene assegnata un priorità, quello con il valore maggiore viene scelto per essere eseguito sulla cpu. In caso di priorità uguale viene sempre usata la metodologia di FCFS.

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2



Figura 5.8: Gaant Scheduling con Priorità.

Le priorità possono essere assegnate internamente, usando informazioni come burst di I/O, burst di CPU, file aperti, memoria, ecc... Alternativamente, possono essere assegnate esternamente, per esempio in base all'importanza del processo. Questo algortimo può essere sia preemtive che nonpreemtive.

Questo algortimo, però, introduce un problema: la **starvation**. Si verifica quanto un processo della ready queue a priorità bassa non viene mai eseguito, questo è probabile in sistemi complessi in cui sono presenti numerosi processi con priorità più alta. Un soluzione adottata è quella dell'**aging**. I processi nella coda di ready possono aumentare la priorità in base al tempo in cui rimangono pronti.

5.4 Scheduling Code Multilivello

Se si usa un algortimo di round robin con una coda a priorità, per trovare i processi con priorità si ha una complessita $O(n)$. Una strategia per ridurre la complessità può essere quella di creare più code, ognuna associato un numero di priorità.

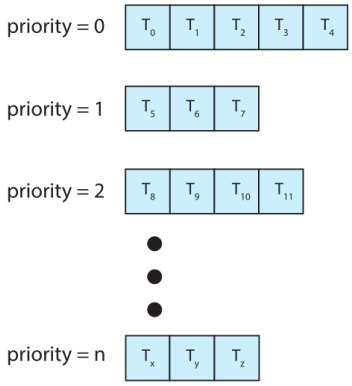


Figura 5.9: Code di Priorità.

Questo meccanismo permette di dividere i processi in diverse categorie, per esempio, *background* e *foreground*, oppure dividendoli in processi *real time*, di *sistema*, *interattivi*, ecc...

5.5 Thread Scheduling

Nelle sezioni precedenti era stata specificata la differenza fra **kernel thread** e **user thread**. I kernel thread vengono gestiti dal sistema operativo, invece, gli user thread sono gesiti da librerie utente. Per il collegamento fra i due tipi di thread sarà necessario avere un processo molto leggero (**LWP**).

5.5.1 Contention Scope

I modelli di gestione dei thread come many-to-many e many-to-one, la libreria dei thread gestisce quali thread assegnare all'LWP, conosciuto anche come **Process-Contention Scope** (**PCS**), dato che i thread devono competere fra di loro. Il meccanismo di scheduling del thread su un core fisico, si chiama **System-Contention Scope** (**SCP**).
Nel caso di modelli one-to-one, il sistema di PCS non viene usato, ma si usa solo il PCS, dove tutti i thread competono fra di loro.

Pthread Scheduling

L'API POISX di `Pthread`, permette di usare entrambi le metodologie:

- **PCS Scheduling:** in `Pthread` si può usare con il valore `PTHREAD_SCOPE_PROCESS`.
- **SCP Scheduling:** in `Pthread` si può usare il valore `PTHREAD_SCOPE_SYSTEM`

C Esempio in C per conper controllare lo SCOPE

```c
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5

int main(int argc, char *argv[]) {
  int i, scope;
  pthread_t tid[NUM_THREADS];
  pthread_attr_t attr;

  /* get the default attributes */
  pthread_attr_init(&attr);

  /* first inquire on the current scope */
  if (pthread_attr_getscope(&attr, &scope) != 0)
    fprintf(stderr, "Unable to get scheduling scope\n");
  else {
    if (scope == PTHREAD_SCOPE_PROCESS)
      printf("PTHREAD_SCOPE_PROCESS");
    else if (scope == PTHREAD_SCOPE_SYSTEM)
      printf("PTHREAD_SCOPE_SYSTEM");
    else
      fprintf(stderr, "Illegal scope value.\n");
  }

  /* set the scheduling algorithm to PCS or SCS */
  pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

  /* create the threads */
  for (i = 0; i < NUM_THREADS; i++)
    pthread_create(&tid[i],&attr,runner,NULL);

  /* now join on each thread */
  for (i = 0; i < NUM_THREADS; i++)
    pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
  /* do some work ... */
  pthread_exit(0);
}
```

5.6 Scheduling per Processi Multipli

Nella prima parte è stato trattato lo scheduling su single-core, invece, sopratutto in sistemi moderni, i sistemi presentano più corse fisici, rendendo più complesso lo scheduling.

5.6.1 Approcci allo Scheduling di Processi Multipli

Un primo approccio usato, chiamato **Multiprocessing Asimetrico**, è quello di dedicare un solo core per le decisioni di scheduling, gestione dell'I/O e altre attività del sistema. Gli altri core eseguono solo il codice dell'utente. Questo, però, può potenzialmente creare un bottlenck.
Nei sistemi moderni si è passato a un approccio diverso, ossia **Multiprocessing Simmetrico** (**SMP**), dove ogni core si auto schedula. Ogni processo, in questo caso, hai il suo scheduler che seleziona da una coda di thread pronti, creando due situazioni diverse:

1. Una coda unica globale con tutti i thread.
2. Una coda di thread privata per ogni processo.

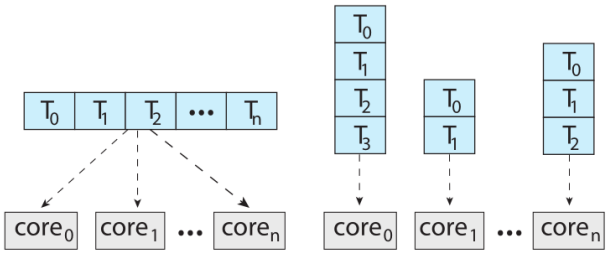


Figura 5.10: Esempio delle due situazioni.

Il primo scenario, potrebbe a sua volta creare un bottleneck, quindi, normalmente è usato sempre il secondo.

5.6.2 Processori Multicore

In sistemi moderni, invece di avere più processori, si è passato ad avere più core nello stesso processore. Permette di essere più veloce e più efficente.
Un problema però, dato che i processori hanno velocità molto elevate, aspettano quasi la metà del tempo ad aspettare le memorie, questo tempo è chiamato **memory stall**.

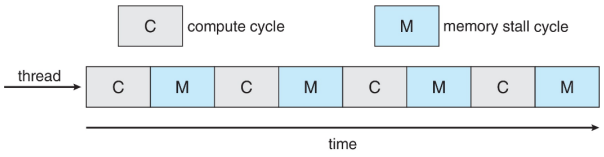


Figura 5.11: Esempio con Memory Stall.

Una soluzione è usare sempre più di un thread, così nel caso uno abbia un memory stall, si sempre l'altro.

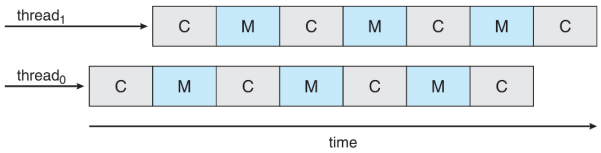


Figura 5.12: Esempio con Memory Stall e più Thread.

Questa tecnica è chiamata **Chip Multithreading** (**CMT**). Per esempio, un processore con 4 core, di cui ognuno ha 2 thread, agli occhi del sistema operativo avrà 8 CPU. Esitono due tipi:

1. **Coarse-Grained:** un core esegue finchè non incontra un evento molto come un memory stall. Quando succede, si cambia a un nuovo thread che inizia l'esecuzione. Cambiare i thread richiede di svutare la pipeline ed è quindi molto costoso.

2. **Fine-Grained:** cambiano i thread con un grado più fino, vicino al confine di un ciclo di istruzione, aggiungendo una logica di thread switching. Questo metodo aggiunge un costo piccolo.

NOTA BENE

Le risorse come cache e pipeline, sono condivise fra tutti gli hardware thread, dei quali ne può essere eseguito solo uno alla volta. Servono, quindi, due livelli di decisione:

1. Primo livello in cui il sistema operativo decide quali thread software devono girare sui thread hardware.
2. Secondo livello in cui viene deciso quale thread hardware deve essere eseguito.

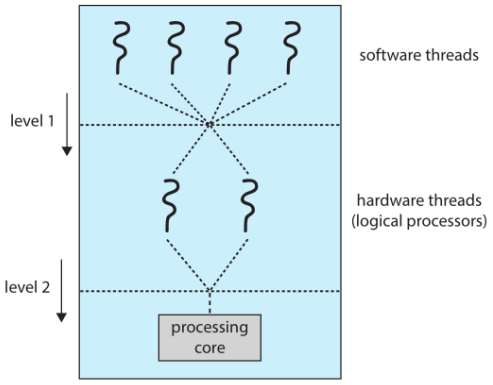


Figura 5.13: Esempio di Multithreading.

DEFINITION Load Balancing

Nei sistemi multicore simmetrici, per sfruttare al meglio e in modo efficente tutti i core, per evitare che alcuni siano in idle e altri con le code piene, è necessario un **load balancer**. Ci sono due approcci diversi:

1. **Push Migration:** un task controlla periodicamente i core, nel caso uno sia sovraccaricato, sposta i thread in un core idle e meno occupato.
2. **Pull Migration:** un core idle prende thread da core più occupati o sovraccaricati.

5.7 Scheduling della CPU in Real Time

Lo scheduling per sistemi real time richiede qualche accortezza in più. In generale, però, possiamo dividerli in due categorie:

1. **Soft Real-Time Systems:** i processi critici hanno priorità maggiore, ma non si sa quando verranno eseguiti.
2. **Hard Real-Time Systems:** i processi **devono** essere eseguiti prima della loro deadline.

5.7.1 Minimizzare la Latenza

I sistemi real time sono basati tipicamente su eventi, quindi quando ricevono una richiesta, è necessario che rispondano nel minor tempo possibile. Il tempo passato da quanto accade un evento e quando viene servito è chiamato **latenza dell'evento**. Questo tempo può cambiare per molti fattori, tra cui:

- **Latenza di Interrupts:** tempo passato fra l'arrivo di una interrupt e l'inizio della sua routine.
- **Latenza di Dispatch:** tempo usato dal dispatcher per cambiare dal processo corrente a un altro processo. In questa latenza, potrebbe pure esserci una fase di conflitto, nel caso sia preemtive.

5.7.2 Scheduling Basato su Priorità

La parte più importante nei sistemi real time è rispondere velocemente, quindi devono poter avere uno scheduling con priorità e preemtive. Questo garantisce sicuramente che il sistema sia almeno soft real-time. Per i sistemi hard real-time, invece, deve essere garantito che finiscano entro la loro deadline.
Prima di capire i var algortimi, è necessario capire le caratteristiche di ogni processo:

1. I processi sono **periodici**. Richiedono la CPU a intervalli di tempo p.
2. Ogni processo a un tempo di fissato di esecuzione t.
3. Ogni processo ha una deadline d entro qui deve essere completato.
4. Il **rate** di una task periodica è $\frac{1}{p}$.

La relazione fra questi valori è:

$$0 \leq t \leq d \leq p$$

Ogni processo, però, deve annunciare allo scheduler il suo tempo di deadline, permettendo allo scheduler due scelte: accettarlo o rifiutarlo, nel caso non possa garantire l'esecuzione entro la deadline.

5.7.3 Rate-Monotic Scheduling (RMS)

In questo algortimo, la priorità è inversamente proporzionale al periodo. Più il periodo è corto, maggiore è la priorità. Questo favorisce i processi che richiedono spesso la CPU per poco tempo.

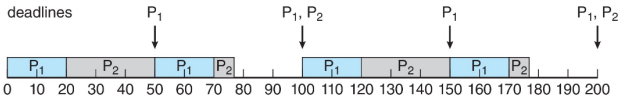


Figura 5.14: Esempio con priorità di P1 magiore di P2.

Alcune volte, però, potrebbe capitare che i processi non finiscano entro la loro deadline.

5.7.4 Ealiest Deadline First Scheduling (EDF)

In questo algortimo la priorità viene assegnata in modo dinamico in base alla deadline, prima è più sarà alta la priorità.
Ogni processo deve annunciare al sistema la propria deadline, dopo le priorità verranno aggiustate in base al nuovo processo.

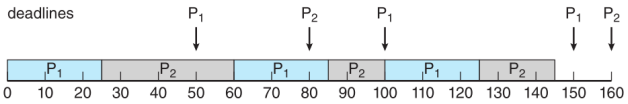


Figura 5.15: Esempio di Algortimo EDF.

5.7.5 Proportional Share Scheduling (PSS)

In questo algortimo alloca T parti fra tutte le applicazioni. Successivamente, un'applicazione può chiedere n parti, avendo quindi $\frac{N}{T}$ del tempo totale del processore.
Aggiungendo delle regole di ammissione e controllo, si riesce a garantire ai processi le parti necessarie per eseguire completamente.

5.7.6 Scheduling Real-Time POSIX

All'interno dell'API standard esistono due tipi di classi che permettono l'implementazione di thread real-time:

- `SCHED_FIFO`: usa una coda FIFO, il primo che entra è il rpimo che esegue.
- `SCHE_RR`: usa un algortimo round-robin che garantisce parti di tempo a ogni task.

Tramite due funzioni è possibile impostare un di queste di policy:

1. `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`.
2. `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`.

C Esempio di Scheduling Real-Time POSIX

```c
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[])
{
  int i, policy;
  pthread_t tid[NUM_THREADS];
  pthread_attr_t attr;

  /* get the default attributes */
  pthread_attr_init(&attr);

  /* get the current scheduling policy */
  if (pthread_attr_getschedpolicy(&attr, &policy) != 0)
    fprintf(stderr, "Unable to get policy.\n");
  else {
    if (policy == SCHED_OTHER) printf("SCHED_OTHER\n");
    else if (policy == SCHED_RR) printf("SCHED_RR\n");
    else if (policy == SCHED_FIFO) printf("SCHED_FIFO\n");
  }

  /* set the scheduling policy - FIFO, RR, or OTHER */
  if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0)
    fprintf(stderr, "Unable to set policy.\n");

  /* create the threads */ for (i = 0; i < NUM_THREADS; i++)
  pthread_create(&tid[i],&attr,runner,NULL);

  /* now join on each thread */ for (i = 0; i < NUM_THREADS; i++)
  pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param){
  /* do some work ... */
  pthread_exit(0);
}
```

Capitolo 6

Strumenti di Sincronizzazione

Nei capitoli precedenti abbiamo visto che più processi possono eseguire parallelamente, crando però diversi problemi, come la **race condition**. Per evitare situazioni come questa, è necessario usare metodi di sincronizzazione o di coordinazione.

6.1 Problema della Sezione Critica

Ipotizziamo di avere n processi $[P_1, P_2, \ldots, P_n]$, in cui ognuno di questi abbia una parte critica, ovvero una parte in cui deve usare dei dati condivisi con altri processi senza che nessuno li modifichi.
Generalmente, è necessario un metodo per sincronizzare i processi, magari mettendo in evidenza l'inizio e la fine di un sezione di codice critica, dividendola così dal codice restante.

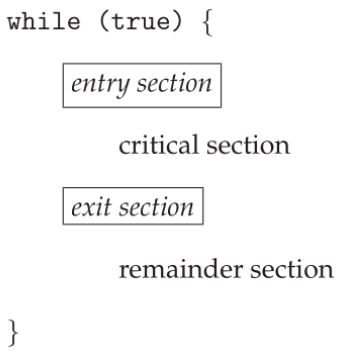


Figura 6.1: Sezione Critica.

Le soluzioni a questo problema devono rispettare queste tre caratteristiche:

1. **Mutual Exclusion:** se il processo P_i sta eseguendo la sua sezione critica, allora nessun altro processo può eseguire la propria zona critica.
2. **Progresso:** se nessun processo sta eseguendo la sua zona critica, ma alcuni vorrebbero, allora la selezione di questo processo non può essere spostata all'infinito.
3. **Bounded Waiting:** deve esistere un limite al numero di volte in cui ad altri processi è consentito entrare nelle proprie sezioni critiche dopo che un processo ha inoltrato una richiesta di accesso alla propria sezione critica.

6.1.1 Prime Soluzioni Software

Nel caso di un sistema singlecore, il problema della sezione critica si può evitare disabilitando l'arrivo delle interrupts, potendo così eseguire il codice in tranquillitaà. Ovviamente, questa soluzione ha diversi limiti, sopratutto il fatto che i computer moderni sono quasi tutti multicore, dove disabilitare le interrupt protrebbe rallentare il sistema. Generalmente, esistono due altri approcci possibili:

1. **Non-Preemtive Kernels:** non permette ai processi che eseguo in kernel mode di essere cambiati con un altro. Questo evita del tutto le race condition.
2. **Preemtive Kernels:** i processi che eseguo in kernel mode possono essere cambiati, creando possibili race condition. I sistemi, però, sono più fluidi e responsivi.

6.1.2 Soluzione di Peternson

Soluzione software classica per risolvere il problema della sezione critica, in particolare questa soluzione risolve il problema di due processi che si alternano nell'esecuzione. Ipotizziamo di avere due processi P_0 e P_1, due varibili condivise `int turn` che indica di chi è il turno e `boolean flag[2]` che indica se il processo è pronto per entrare nella sezione critica. Inoltre, si ipotizza che le istruzioni di load e di store siano atomiche:

C Esempio per P_i (P_0)

```c
while (true){
  flag[i] = true;
  turn = j;

  while (flag[j] && turn == j){}; // wait for turn i
    /* here is the critical section */

  flag[i] = false;
  /* here is the remainder section */
}
```

Correttezza della soluzione

Per verificare che la soluzione si corretta:

1. **Mutual Exclusion:** P_i può entrare solo se è il suo turno o se P_j non è pronto per entrare. **OK**.
2. **Prograss:** P_i può essere bloccata in loop dall'entrare solo se P_j è pronto ed è il suo turno, altrimenti, senza una di queste due condizioni può entrare. **OK**.
3. **Bounded Waiting:** stesso motivo di prima. **OK**.

In archietture moderne, dove le istruzioni senza dipendenze vengono riordinate, si potrebbero creare diversi problemi, come la situazione in cui due processi entrino entrambi nella loro sezione critica.

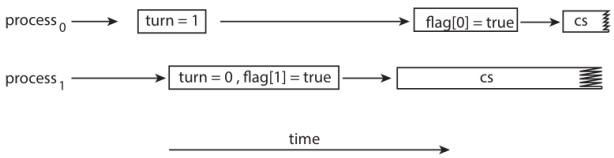


Figura 6.2: Sezione Critica Simultanea.

Questo problema si può risolvere con soluzione hardware, come la **memory barrier**.

6.1.3 Memory Barrier

Di solito quello che garantisce cosa la memoria provvederà a una applicazione è chiamato **modello della memoria**, ne esitono di due tipi:

1. **Strongly Ordered:** quando la modifica della memoria da parte di un processo è immediatamente visibile agli altri processi.
2. **Weakly Oredered:** quando la modifica della memoria da parte di un processo può non essere immediatamente visibile agli altri processi.

Varia a seconda del tipo di processore, quindi non possono essere fatte assunzioni a livello kernel.
Per questo, per evitare problemi, vengono create delle istruzioni che forzano che un cambiamento della memoria si propaghi su tutti i processi.

DEFINITION Memory Barrier

Conosciuta anche come **memory fence**, è un'istruzione che impone che qualsiasi modifica in memoria

venga propagata a tutti gli altri processori. In caso di riordinamento delle istruzioni, questa tecnica permette di di assicurare che le azioni vengano mostrate a tutti i processi.

Questo permette a tutte le istruzioni di load e store di essere completate e visibili.

C Esempi con Thread 1 e 2

```
# Thread 1
while (!flag)
  memory barrier();
print x;

# Thread 2
x = 100;
memory_barrier();
flag = true;
```

6.1.4 Istruzioni Hardware

Molti sistemi moderni mettono a disposizione delle istruzioni a livello hardware che permetto di modificare o sostituire il contenuto di una variabile atomicamente, ossia senza che possa essere fermata per nessun motivo.

Esistono due istruzioni principali:

1. `test_and_set()`.
2. `compare_and_swap()`.

C test_and_set()

```
boolean test_and_set(boolean *target) {
  boolean rv = *target;
  *target = true;

  return rv;
}
```

C compare_and_swap()

```
do {
  while (test_and_set(&lock))
    ; /* do nothing */
  /* critical section */
  lock = false;
  /* remainder section */
} while (true);
```

Questo esempio assicura sicuramente l'esclusione mutua, ma è possibile pure garantire il progresso e il bounded waiting.

C Esempio per Bounded Waiting

```
while (true) {
  waiting[i] = true;
  key = 1;
  while (waiting[i] && key == 1){
    key = compare_and_swap(&lock,0,1)
```

```
  }

  waiting[i] = false;
  /* here is the critical section */
  j = (i + 1) % n;

  while ((j != i) && !waiting[j]){
    j = (j + 1) % n;
  }

  if (j == i)
    lock = 0;
  else
    waiting[j] = false;

  /* here is the remainder section */
}
```

All'interno dell'ISA x86 esiste lo statement `cmpxchg` con il prefisso `lock`, per implementare l'istruzione `compare_and_swap()` in modo atomico. La forma generale risulta del tipo:

```
lock cmpxchg <destination operand>, <source operand>
```

DEFINITION Variabili Atomiche

Le variabili atomiche consentono aggiornamenti atomici, quindi non interrompibili, su tipi di dati di base, come interi e booleani.

Solitamente, istruzioni come `compare_and_swap()`, non vengono usate direttamente, ma piuttosto servono a creare costrutti utili a risolvere il probelma delle sezioni critiche.
Immaginiamo di avere una variabile atomica `sequence` e una funziona che che opera su varibaili atomiche `increment()`:

C Esempio di uso di Varibaili Atomiche

```c
void increment(atomic_int *v){
  int temp;
  do {
    temp = *v;
  } while (temp != (compare_and_swap(v,temp,temp+1)));
}
```